UTF-16 sees more widespread use: Windows until very recently used UTF-16 internally, and the ECMAScript standard (from which all JavaScript engines are derived) specifies a UTF-16 compatible internal format for string data. UCS-2, UCS-4, UTF-16, and UTF-32 all shared a common problem: none of them can be compatible with ASCII, because the minimum size of their code units is two or four bytes.

How About We… Don't Break ASCII, Maybe?

In order for ASCII-encoded data to be readable by a system using variable-width encoding, logic dictates that the minimum code unit size has to be one byte. An attempt was made by the ISO, in a draft of ISO-10646, to make such a transformation format. UTF-1 proposed a minimum of a single byte and up to four bytes to provide the codepoints necessary to meet the Unicode specification. UTF-1 (which you've probably never heard of until now) is consigned to the dumpster of failed standards proposals because of one glaring flaw: it did not protect the existing and widely used ASCII standard.

UTF-1 describes special byte values called *continuation bytes*, which are the valid values for second and subsequent bytes of a multi-byte sequence (where the leading byte in the sequence also has a value from a certain range). For UTF-1, if a byte value of 0xA0 or greater is read, it means that the next two to four bytes are also part of the code unit. In this way it is possible to support millions of codepoints but keep the most commonly used characters within two or fewer bytes. The table below shows all of the UTF-1 valid byte values for single- and two-byte codepoints (sequences with three or more bytes have been omitted).

UTF-1 (Partial)

Table 8-3. *UTF-1 codepoint ranges*

First codepoint	Last codepoint	Byte 0	Byte 1
U+0000	U+009F	00–9F	
U+00A0	U+00FF	A0	A0–FF
U+0100	U+4015	A1–F5	21–7E, A0–FF